

Jest 是由 Meta (Facebook) 開發並維護的一個 JavaScript 測試框架

```
npm i -D jest
```

```
{
  "scripts": {
    "test": "node --disable-warning=ExperimentalWarning --experimental-vm-modules node_modules/jest/bin/jest.js"
  },
  "type": "module",
  "devDependencies": {
    "jest": "^30.4.2"
  }
}
```

```
npm run test
```

<https://regex101.com/>

常規表示法

Regular Expression	Test String
<code>:/[0-9]</code>	admkn mKDM01KMCS 0000000
<code>:/\d</code>	admkn mKDM01KMCS 0000000
<code>:/\w</code>	admkn mKDM01KMCS 0000000
<code>:/\d{2}</code>	admkn mKDM01KMCS 0000000
<code>:/.</code>	admkn mKDM01KMCS 0000000
<code>:/.{7}</code>	admkn mKDM01KMCS 0000000

<https://stackoverflow.com/questions/201323/how-can-i-validate-an-email-address-using-a-regular-expression>

Email的常規表示法

```
(?:[a-z0-9!#$%&'*\x2f=?^_`~\x7b-\x7d~\x2d]+(?:\. [a-z0-9!#$%&'*\x2f=?^_`~\x7b-\x7d~\x2d]+)*|"(?:[\x01-\x08\x0b\x0c\x0e-\x1f\x21\x23-\x5b\x5d-\x7f]|\\[\x01-\x09\x0b\x0c\x0e-\x7f])*")@(?:(?:[a-z0-9](?:[a-z0-9\x2d]*[a-z0-9])?\.|)+[a-z0-9](?:[a-z0-9\x2d]*[a-z0-9])?|\\[(?:(?:(2(5[0-5]| [0-4][0-9]))| 1[0-9][0-9]| [1-9]?[0-9]))\\.)}{3}(?:(2(5[0-5]| [0-4][0-9]))| 1[0-9][0-9]| [1-9]?[0-9])| [a-z0-9\x2d]*[a-z0-9]:(?:[\x01-\x08\x0b\x0c\x0e-\x1f\x21-\x5a\x53-\x7f]|\\[\x01-\x09\x0b\x0c\x0e-\x7f])+)\\))
```

git reset

讓分支 Git 回到過去某一刻，順便決定
要不要保留修改

使用情境：

1. commit 了，但是 commit 的檔案內容不是我要的
2. commit 了，但是 commit 訊息不是我要的 參考
3. 我全都不要了（檔案內容+訊息）



至少要有兩次commit才可以做到「回到上一個commit」
Git reset SHA值後，mixed和soft不會刪掉檔案和修改，但是hard會刪掉

git reset 基本指令 - 1

指令	說明
<code>git reset ^{SHA}<commit></code>	回到（過去的）某個 commit（預設為 --mixed）
<code>git reset --mixed <commit></code>	所有異動的檔案變回 unstaged（add 前）
<code>git reset --soft <commit></code>	所有異動的檔案變回 staged（add 後）
<code>git reset --hard <commit></code>	所有異動的檔案全部還原（謹慎使用）
<code>git reset <檔名></code>	舊用法，將某個檔案變回 unstaged （從 add 後，變為 add 前）

功能等於這行：`git restore --staged aaa.md`

模式名稱 	HEAD 指針 移動	暫存區 (Staging Area)	工作目錄 (Working Directory)	常用場景描述
<code>--soft</code>	移至指定 Commit	保留 (原狀態不變)	保留 (實體檔案不變)	只是想拆掉 Commit 重新包裝。
<code>--mixed</code> (預設)	移至指定 Commit	重置 (清空暫存區)	保留 (實體檔案不變)	想把 Commit 拆掉，檔案退回還沒 <code>add</code> 的狀態。
<code>--hard</code>	移至指定 Commit	重置 (完全清空)	覆蓋 (直接刪除修改)	做實驗完全寫爛了，想徹底放棄所有修改。

git reset 基本指令 - 2

指令	說明
<code>git reset HEAD^</code>	回到上一個 commit ($\wedge$ = 往上一層)
<code>git reset HEAD~3</code>	回到前 3 個 commit ($\sim n$ = 往前 n 步)
<code>git reset <commit>^</code>	回到某 commit 的上一步
<code>git reset <commit>~3</code>	回到某 commit 的前幾步

Q1: `git reset <檔名>` & `git restore --staged <檔名>` 差在哪？

Q2: `git reset 1234` & `git checkout 1234` 差在哪？

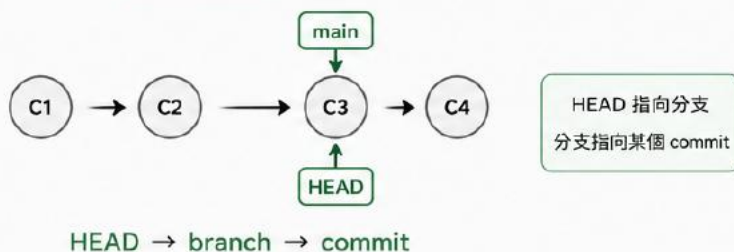
Q3: 當我要 reset 到一個不是我現在分支上的 commit 時，會發生什麼事？

git reflog 基本指令

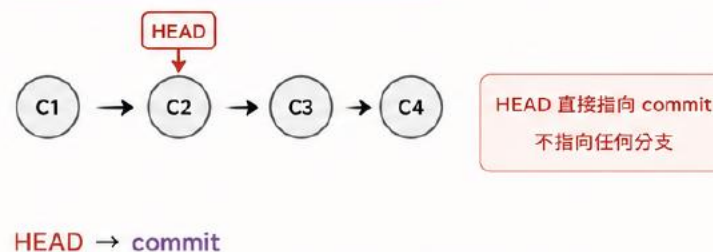
指令	說明
<code>git reflog</code>	每一次 HEAD 移動
<code>git reflog <分支名></code>	看某個分支的移動紀錄
<code>git reflog -n 5</code>	顯示最近 5 筆 (number)
<code>git reflog --date=local</code>	你在什麼時間點，讓 HEAD 做了什麼變動

什麼是 detached HEAD

正常狀態 (HEAD 指向分支)

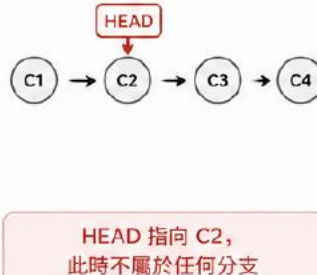


detached HEAD 狀態 (HEAD 直接指向 commit)

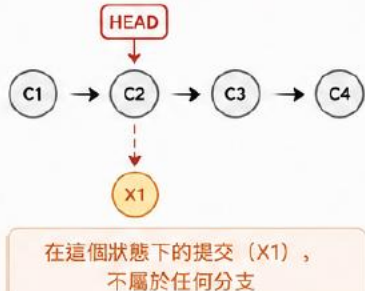


當你在 detached HEAD 狀態下提交，並切換分支後會發生什麼？

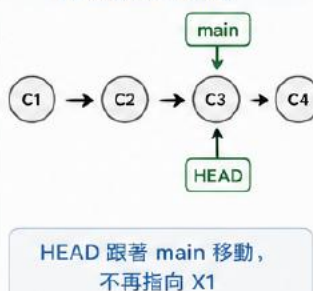
1 進入 detached HEAD (checkout 某個 commit)



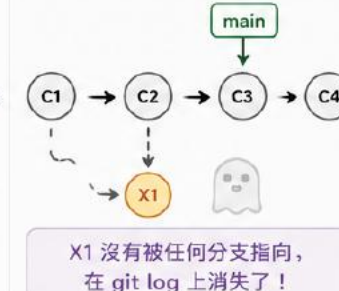
2 在 detached HEAD 狀態下新提交



3 切換回 main 分支 (checkout main)



4 查看 git log 的結果



detached HEAD 斷頭狀態

如果我不小心checkout到某個SHA值上並且變動檔案，已經add和commit，等於我在不屬於我的branch上commit了(就是我根本沒有在任何branch上commit)。

如果我想保留剛剛的修改，並安全回到 main

解決辦法：

1. 強制將 main 分支的標籤移動到新做好的 SHA 值上：git branch -f main <新SHA值>
2. 這時 main 已經移過來了，但我還在斷頭狀態，需切回 main分支：git checkout main

<https://ithelp.ithome.com.tw/articles/10238939>

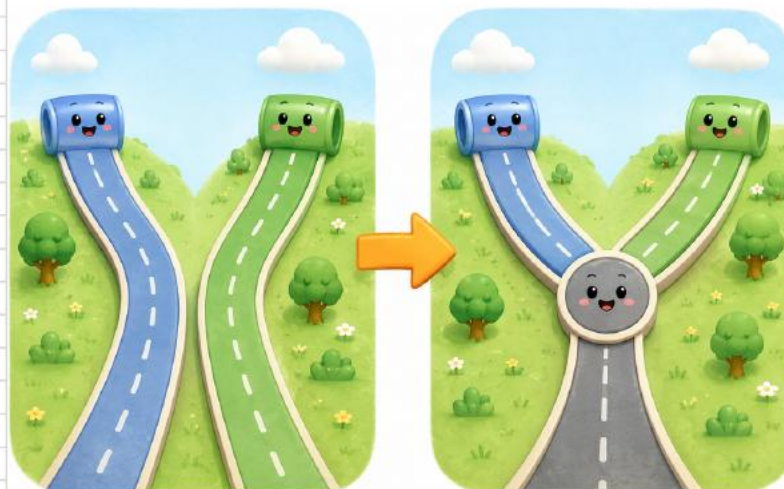
git merge <分支名>

將另一個分支的變更合併進「當前的分支」

例如：團體協作開發完功能要合併回主分支

正常來說 merge 會多一個節點（訊息有預設，也可在編輯器自訂）

如果分支沒有分叉，是「直線前進」，會直接快轉（fast-forward），不會有節點



git merge 基本指令

指令	說明
<code>git merge <分支名></code>	將某分支合併到目前分支
<code>git merge -ff <分支名></code>	允許 fast-forward (預設行為) 同上
<code>git merge --no-ff <分支名></code>	強制建立 merge commit (不用 fast-forward)
<code>git merge --abort</code>	取消 merge (發生衝突時)
<code>git merge --no-ff -m "message" <branch></code>	強制建立 merge commit, 並指定訊息 (不會進到 vim 編輯器)

當遇到會產生新節點的Y字型情況，執行完 `git merge molly`(分支名) 之後，會跳出問我要不要改備註訊息的vim編輯器，如果我想改就先打小寫 `i` 進入編輯模式，改好之後按 `esc` 退出編輯，再打 `:wq` 就能做到儲存並離開。

vim 編輯器

vim 是跑在終端機裡的文字編輯器。

Git 預設編輯器是 vim，要輸入 commit message 時會自動開啟

指令

說明

`i`

進入編輯模式（才能打字）

`esc`

離開編輯模式，回到指令模式

`:w`

儲存 (write)

`:q`

未修改時離開 (quit)

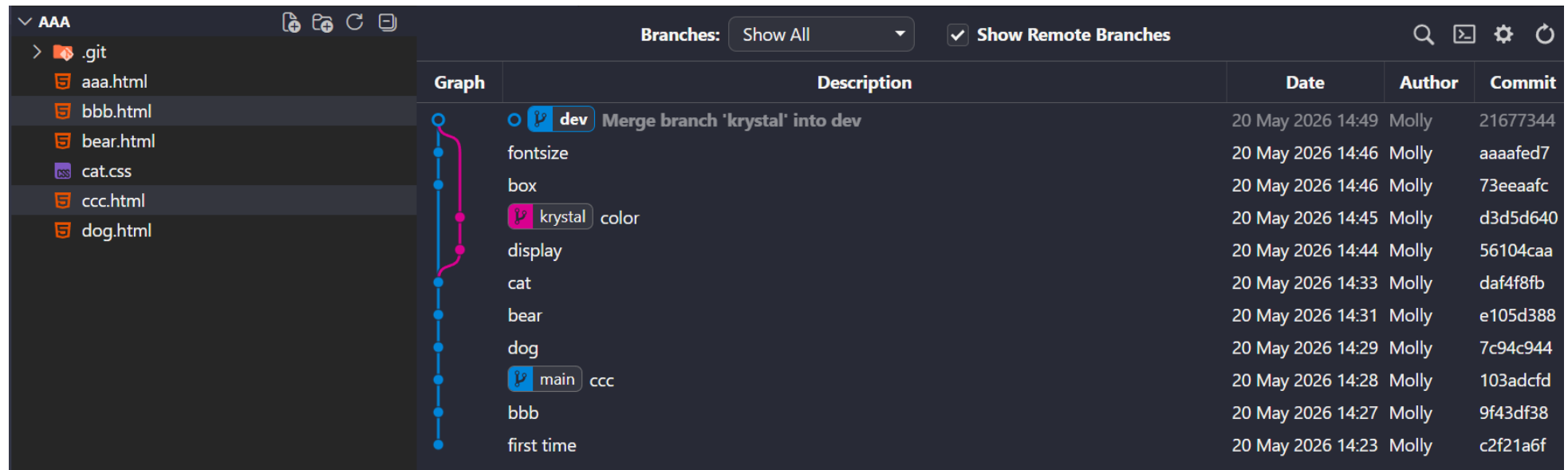
`:wq`

儲存並離開

有merge commit的merge練習

```
git add .
git commit -m "bbb"
git add .
git commit -m "ccc"
git checkout -b dev
git add .
git commit -m "dog"
git add .
git commit -m "bear"
git branch krystal dev
git checkout krystal
git add .
git commit -m "cat"
git checkout dev
git merge krystal
git checkout krystal
git add .
git commit -m "display"
git add .
git commit -m "color"
git checkout dev
git add .
git commit -m "box"
git add .
git commit -m "fontsize"
git merge krystal
```

代表我站在dev 要把krystal合併進來



merge 解衝突

git reset aaaafed7 (為了練習先回到上個練習的merge前)
取消前面檔案的變更(左側變更欄會顯示刪除線的檔案)

此時人在dev

git add .

git commit -m "fox"

其實這個練習不用新增fox是我多做了但也沒差

git checkout krystal

在aaa.html新增一個p

git add .

git commit -m "kp"

git checkout dev

在aaa.html新增一個div

git add .

git commit -m "devdiv"

git merge krystal

因為兩個位置都在同個檔案做更動

所以產生了衝突

可以擇一保留或兩者都要或都不要

選完記得ctrl+s存檔

存檔前也可以自己打字更動內容

然後一樣要記得add、commit

git add .

git commit -m "解決衝突"

```
原始檔控制
變更
Merge branch 'krystal' into dev
合併變更 1
aaa.html +
暫存的變更 1
變更 0

aaa.html > html > body > ?
2 <html lang="en">
3 <head>
7 </head>
8 <body>
9 <<<<<< HEAD (目前變更)
10 <div>在dev新增這個div</div>
11 =====
12 <p>在krystal新增這個p</p>
13 >>>>>> krystal (來源變更)
14 </body>
15 </html>
16

問題 輸出 偵錯主控台 終端機 連接埠 GITLENS

PS C:\Users\Galaxy\Desktop\aaa> git add .
PS C:\Users\Galaxy\Desktop\aaa> git commit -m "fox"
[dev 641e271] fox
1 file changed, 3 insertions(+)
create mode 100644 fox.css
PS C:\Users\Galaxy\Desktop\aaa> git checkout krystal
Switched to branch 'krystal'
PS C:\Users\Galaxy\Desktop\aaa> git add .
PS C:\Users\Galaxy\Desktop\aaa> git commit -m "kp"
[krystal 0c523ca] kp
1 file changed, 3 insertions(+), 1 deletion(-)
PS C:\Users\Galaxy\Desktop\aaa> git checkout dev
Switched to branch 'dev'
PS C:\Users\Galaxy\Desktop\aaa> git add .
PS C:\Users\Galaxy\Desktop\aaa> git commit -m "devdiv"
[dev bdf6f8a] devdiv
1 file changed, 3 insertions(+), 1 deletion(-)
PS C:\Users\Galaxy\Desktop\aaa> git merge krystal
Auto-merging aaa.html
CONFLICT (content): Merge conflict in aaa.html
Auto-merging cat.css
Automatic merge failed; fix conflicts and then commit the result.
PS C:\Users\Galaxy\Desktop\aaa>
```

原始機控制

變更

訊息 (要在 "Ctrl+Enter" 上提交 dev)

發佈 Branch

圖表

解決衝突 Molly

kp Molly

color Molly

display Molly

devdiv Molly

fox Molly

fontsize Molly

box Molly

cat Molly

bear Molly

dog Molly

ccc Molly

bbb Molly

first time Molly

GITLENS

Git Graph

Description

dev 解決衝突

devdiv

krystal kp

fox

fontsize

box

color

display

cat

bear

dog

main ccc

問題 輸出 偵錯主控台 終端機 連接埠 GITLENS

```
PS C:\Users\Galaxy\Desktop\aaa> git checkout krystal
PS C:\Users\Galaxy\Desktop\aaa> git add .
PS C:\Users\Galaxy\Desktop\aaa> git commit -m "kp"
[krystal 0c523ca] kp
1 file changed, 3 insertions(+), 1 deletion(-)
PS C:\Users\Galaxy\Desktop\aaa> git checkout dev
Switched to branch 'dev'
PS C:\Users\Galaxy\Desktop\aaa> git add .
PS C:\Users\Galaxy\Desktop\aaa> git commit -m "devdiv"
[dev bdf6f8a] devdiv
1 file changed, 3 insertions(+), 1 deletion(-)
PS C:\Users\Galaxy\Desktop\aaa> git merge krystal
Auto-merging aaa.html
CONFLICT (content): Merge conflict in aaa.html
Auto-merging cat.css
Automatic merge failed; fix conflicts and then commit the result.
PS C:\Users\Galaxy\Desktop\aaa> git add .
PS C:\Users\Galaxy\Desktop\aaa> git commit -m "解決衝突"
[dev 86886cd] 解決衝突
PS C:\Users\Galaxy\Desktop\aaa>
```

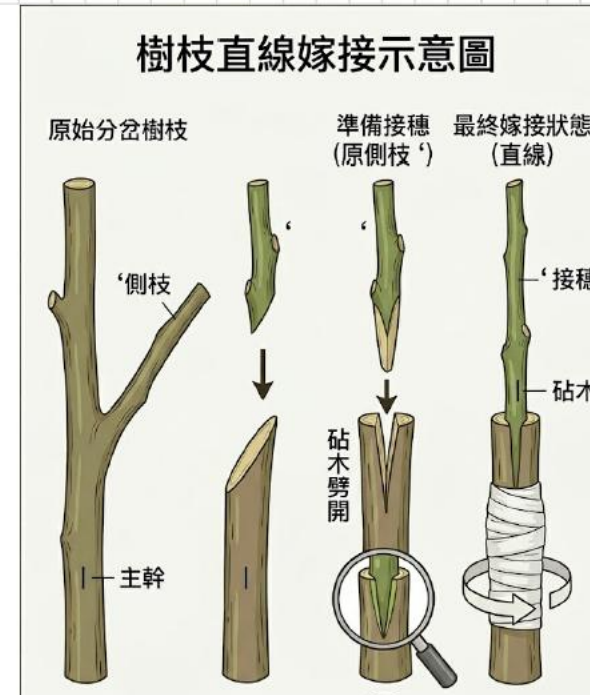
如前一頁的說明
一樣要記得add、commit
git add .
git commit -m "解決衝突"
做完之後git graph就會長這樣

git rebase <分支名>

將「目前分支」的變更，搬到另一個分支
的最新位置上

例如：團隊更新了主分支，你要把自己的
功能「接到最新主分支後面」

rebase 會改變 commit 歷史（重新產生
commit）



git rebase 基本指令

指令

`git rebase <分支名>`

`git rebase --continue`

`git rebase --abort`

`git rebase --skip`

說明

將目前分支嫁接到某分支之後

解決衝突後，繼續解下一個 commit 的衝突

取消 rebase (回到原本狀態)

跳過解當前 commit 的衝突

Q1: merge & rebase 的差別?

Q2: merge & rebase 用哪個好?

前面的例子都是dev會跑到最上面

現在要先建立melissa分支

分別在dev和melissa做commit

然後在melissa把melissa分支嫁接到dev之後(melissa會在最上面)

那這樣和在melissa執行git merge dev(melissa也會在最上面)有何不同?

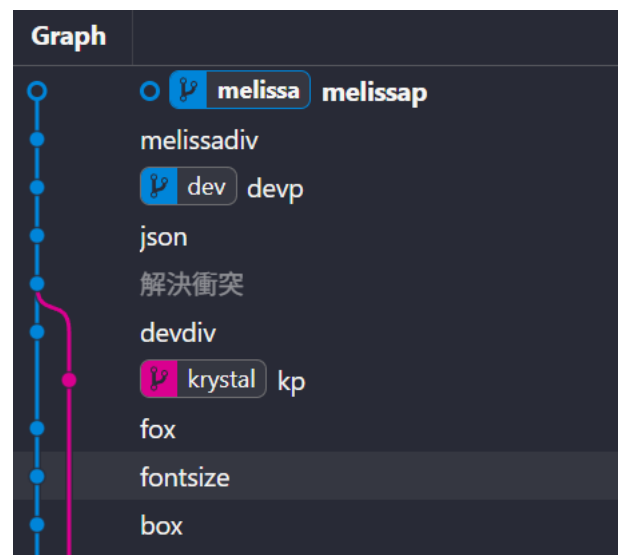
用rebase的優點：歷史紀錄極度乾淨、線性，便於團隊進行 Code Review(不會有太多線)。

缺點：接上去的 Commit 雖然內容一樣，但 Commit ID (Hash 值) 全都變了，本質上它們是全新的 Commit。

一般的rebase嫁接練習

```
git branch melissa  
git checkout melissa  
git add bbb.html  
git commit -m "melissadiv"  
git add ccc.html  
git commit -m "melissap"  
git checkout dev  
git add bear.html  
git commit -m "devp"  
git checkout melissa  
git rebase dev
```

代表我站在melissa 要把melissa嫁接到dev之後(之後就是最上面的意思)



rebase解衝突

現在要先建立andy分支

分別在dev和andy做commit(兩者要對同一個檔案做更動才会有衝突)

然後在andy把andy分支嫁接到dev之後(之後就是最上面的意思)

rebase解決衝突的方式和merge不太一樣

因為rebase的嫁接方式是把andy的commit一顆一顆丟過去dev比對衝突

解決衝突過程一樣是擇一保留或兩者都要或都不要

選完記得ctrl+s存檔

存檔前也可以自己打字更動內容

一樣要記得add

這邊開始不一樣：不用輸入git commit -m “解決衝突”的指令

而是輸入git rebase --continue

然後就可以繼續檢查下一個commit的衝突

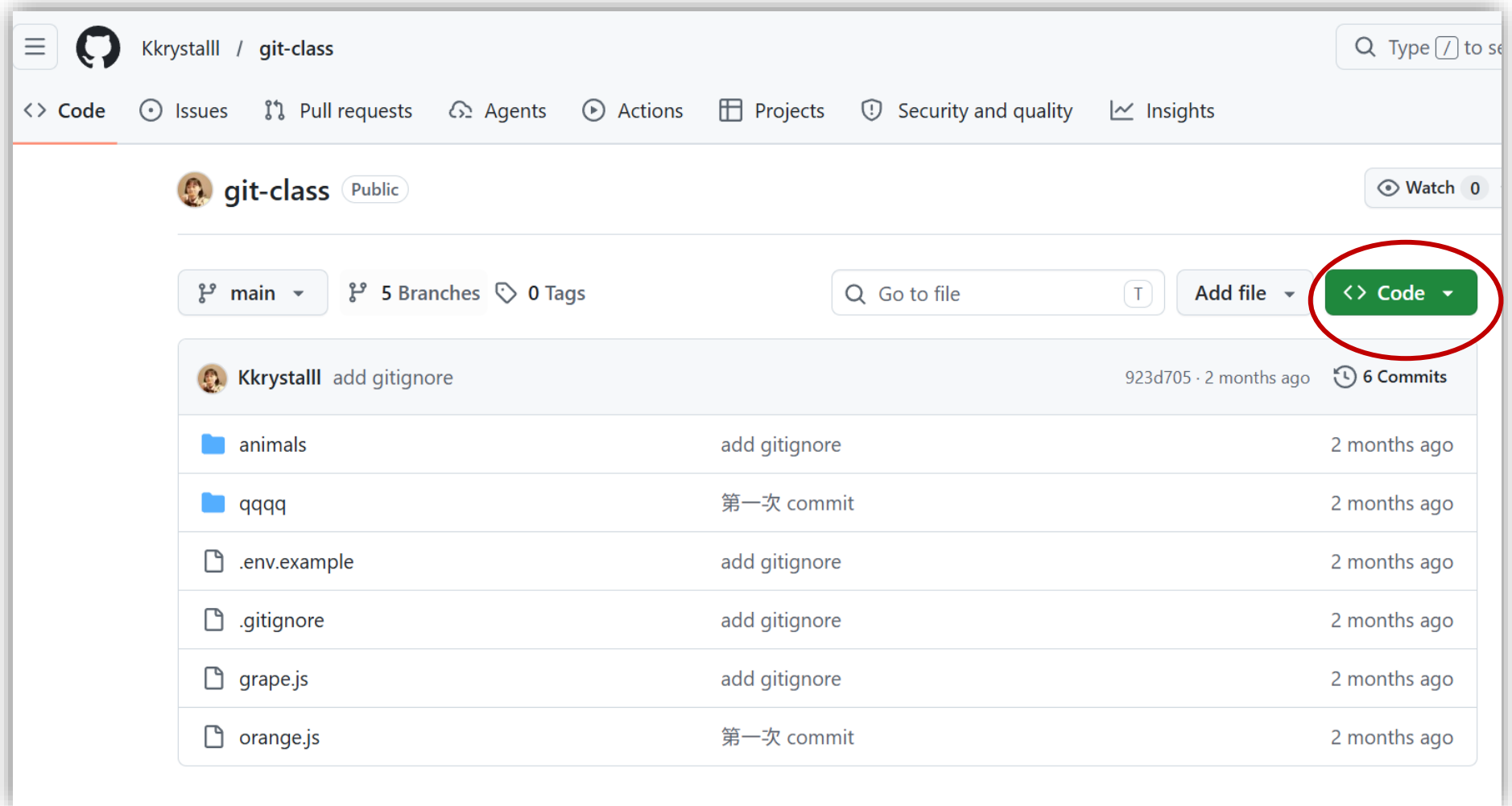
git clone <repo-url>



把遠端 repository 複製一份到本地
(把 GitHub 專案整包抓到我本地開始用)

- ✓ 會做的事情
 - 下載整個專案
 - 自動建立 .git
 - 自動設定遠端 origin

1. 開電腦終端機
2. 然後cd到資料夾或桌面
3. 再輸入 `git clone` 網址(網址在GitHub)
4. 開VS Code把資料夾抓進去就成功啦



查看目前所在的專案資料夾連接了哪些遠端儲存庫：\$git remote -v

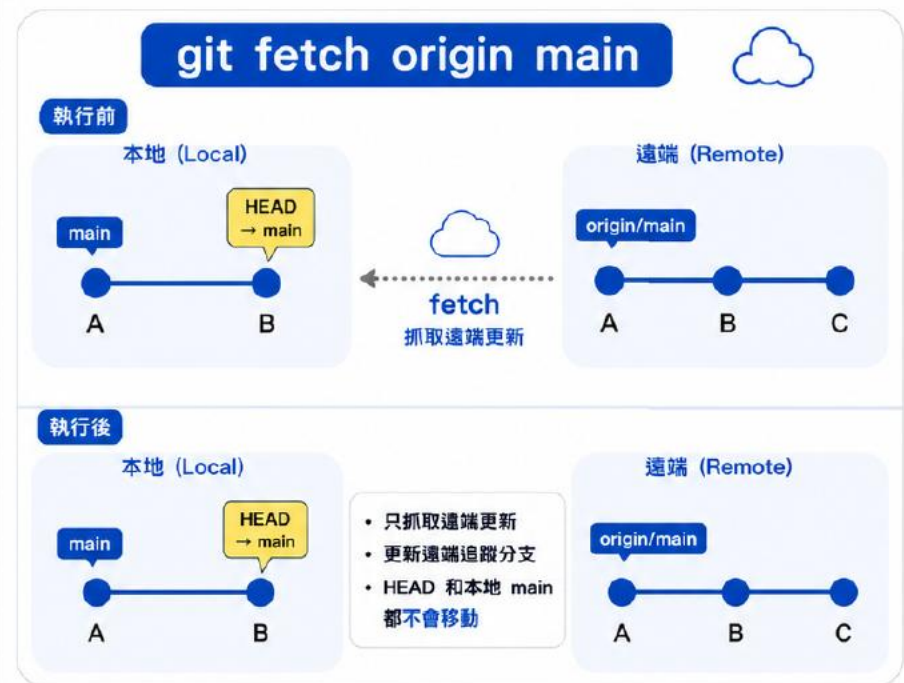
git fetch <儲存庫名> <分支名>

抓取遠端最新資料，但不會影響目前檔案
(HEAD 不會移動，目前工作中的分支
也不會改變)

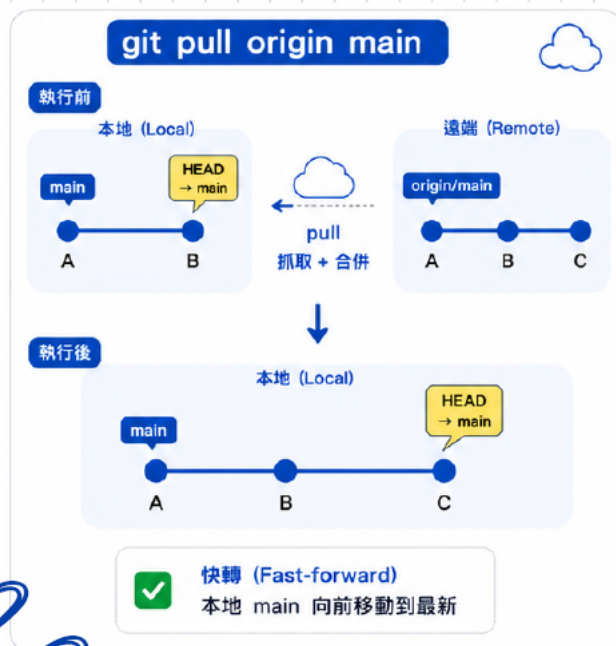
✓ 會做的事情

- 更新遠端分支 (origin/*)
- 不會改你的程式碼

Show Remote Branch 要勾才會看得到



git pull <儲存庫名> <分支名>



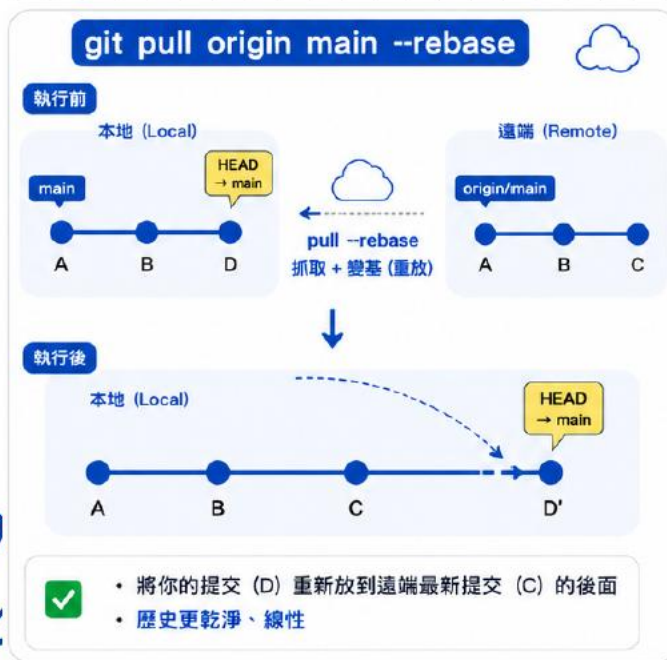
抓取遠端更新，並直接合併到目前分支
等同於 `git fetch` + `git merge`

✓ 會做的事情

- 更新遠端分支 (origin/*)
- 更新目前所在的本地分支 (HEAD 會移動)
- 有分叉時會產生 merge commit

Q: 為何 `git pull` 沒有生成新 commit ?

git pull <儲存庫名> <分支名>



git pull <儲存庫名> <分支名> --rebase
抓取遠端更新，並將本地的 commit 接到遠端之後

等同於 git fetch + git rebase

✓ 會做的事情

- 更新遠端分支 (origin/*)
- 更新目前在本地的分支 (HEAD 移動)
- 有分叉時自動 rebase，不產生 merge commit

origin/dev不是遠端的最新紀錄而是本地的遠端紀錄

開 issues

- 確認沒有重複的 issue（可由一人統一開票）
- 標題簡短清楚，可加點數或分類
- 描述清楚問題、預期行為、實際行為，附截圖或錯誤訊息
- 用 Label 標註性質，Milestone 管理階段目標
- 每人一次只接一張，assign 給自己
- 完成後關閉 issue（手動或 PR 加 close）





Molly-0808 / my_md

🔍 Type to search



<> Code

⦿ Issues

🔗 Pull requests

🤖 Agents

▶ Actions

📅 Projects

📖 Wiki

🛡️ Security and quality

📈 Insights

⚙️ Settings

is:issue state:open



Labels



Milestones

New issue



Open

0

Closed

0

Author ▾

Labels ▾

Projects ▾

Milestones ▾

Assignees ▾

🔼🔽 Newest ▾

No results

Try adjusting your search filters.

[Give feedback](#)

Add a description

Write

Preview

H B I       @   ↶

- 使用者可以輸入帳號、密碼

```

```

- 驗證帳號跟密碼對不對的起來

- 按下登入按鈕之後 (form 表單)

- 會跳轉頁面 `/home`

- cake 按鈕 [連結到這個網址](https://www.cake.me/users/sign-in)

☐ Open 2 Closed 0

☐  [Bug] 按下登入會壞掉 (1)

#3 - Kkrystalll opened now

☐  [切版] 登入功能 (1)

點數

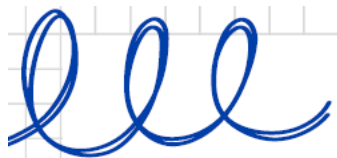
#2 - Kkrystalll opened 5 minutes ago

☐  [功能] 登入功能 (2)

#1 - Kkrystalll opened 20 minutes ago

用labels做 不用手動打出來

Pull Request 的本質是「請求」目標專案合併你「已經存在於 GitHub 上」的某個分支



Pull requests

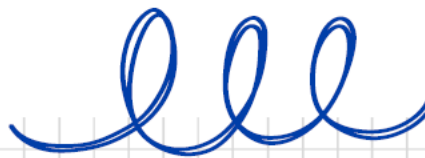


為何需要發 PR

- 讓團隊審查程式碼，減少錯誤
- 留下修改紀錄

發 PR 流程

- 從自己分支發 PR 到目標分支
- 填寫標題、描述、標記 issue
- 依回饋修改，通過後 merge or rebase



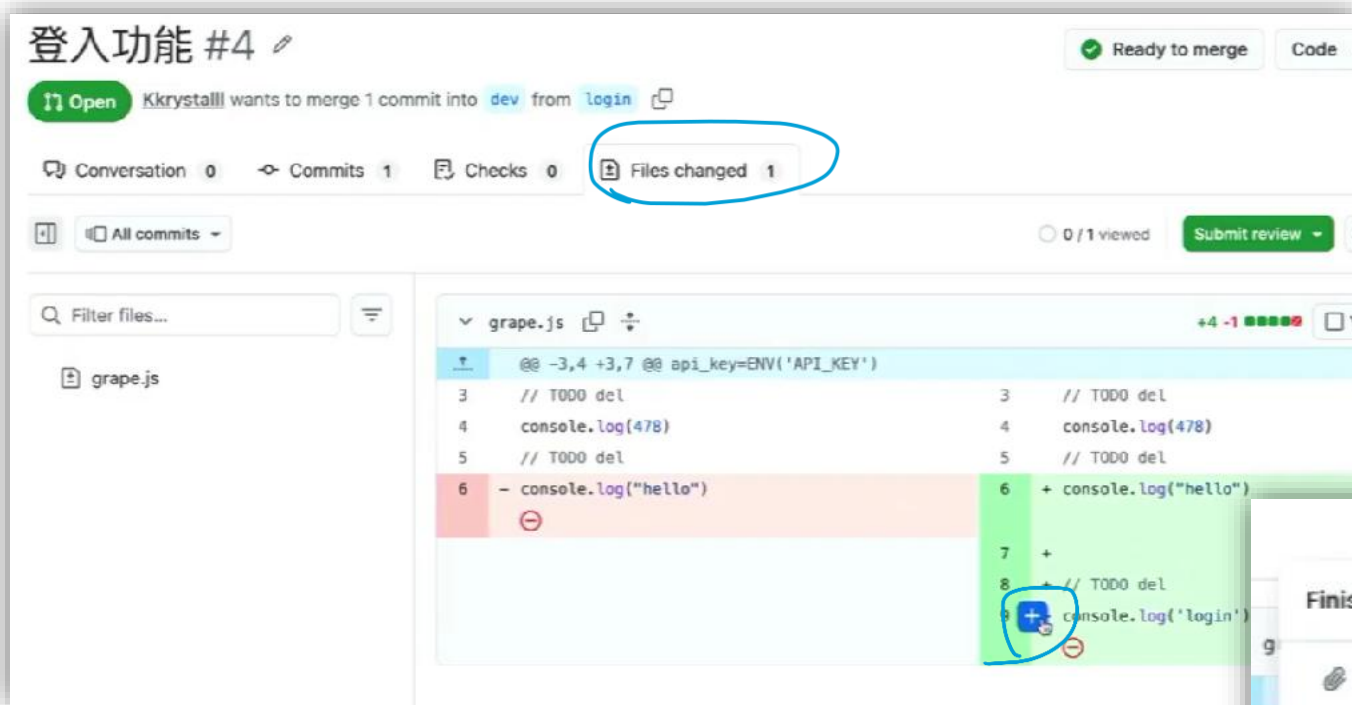
發PR前要做：

1. 先確認HEAD位置，要在我開發或修改檔案的分支
2. 把最新遠端pull下來看有沒有衝突，如果有衝突，就在編輯器裡修好，然後 commit
3. 確認過再推上去： `git push origin <分支名稱>`
4. 再去發PR請求合併



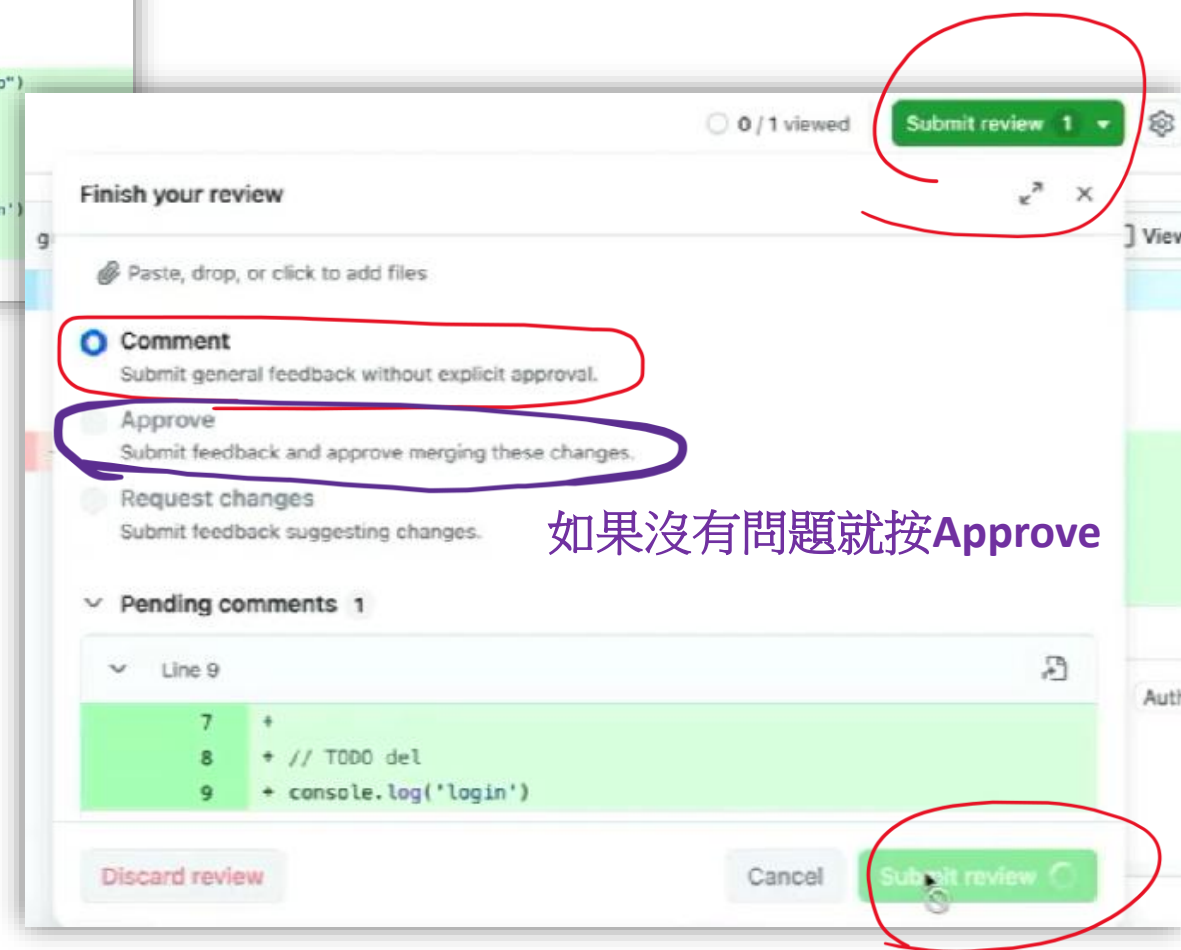
解衝突步驟

1. checkout到本地dev
2. 把遠端dev最新進度拉下來：git pull origin dev
3. checkout到我開發或修改的分支，比如login
4. 把login嫁接到dev後面：git rebase dev
5. 解衝突 保留兩者/擇一/捨棄兩者
6. Ctrl+S
7. git add 檔名
8. git rebase --continue
9. 終端機輸入：「:wq」
10. 解衝突好了之後，本地login分支會在最上面，但是遠端login還在下面，導致推不上去
11. 這時就必須強推，將本地 login 推到遠端 origin ：git push origin login -f

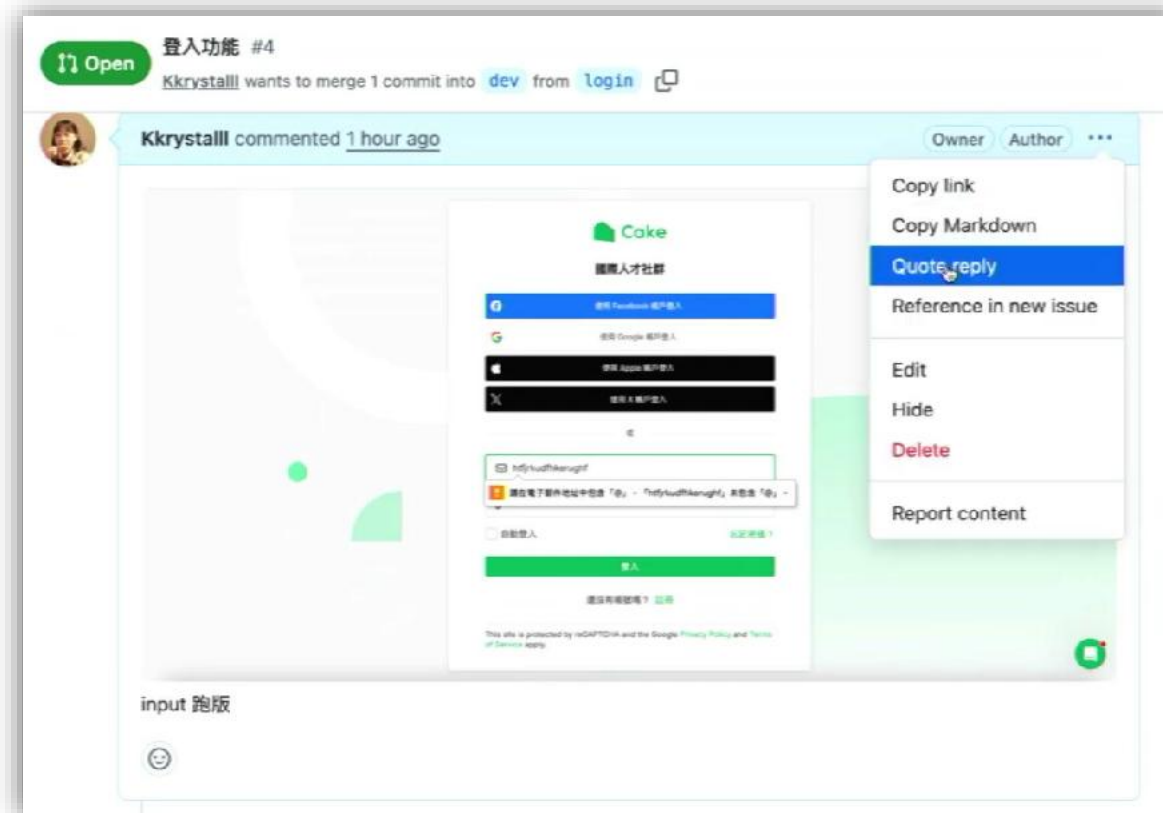


如果有檢查到問題要留言

超重要！留言好存檔不算完成，
要按上面的submit review
選擇Comment



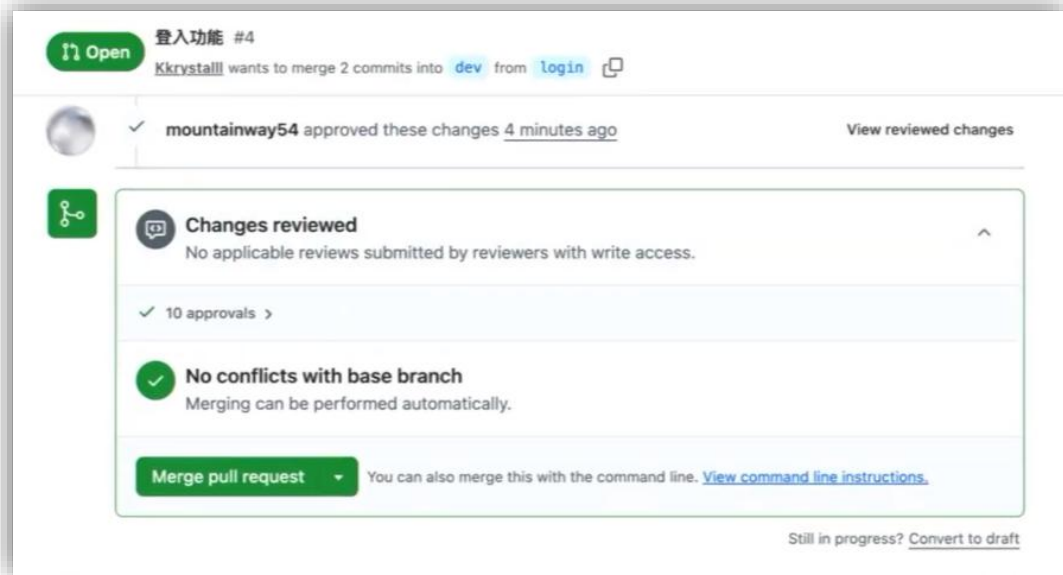
如果沒有問題就按Approve



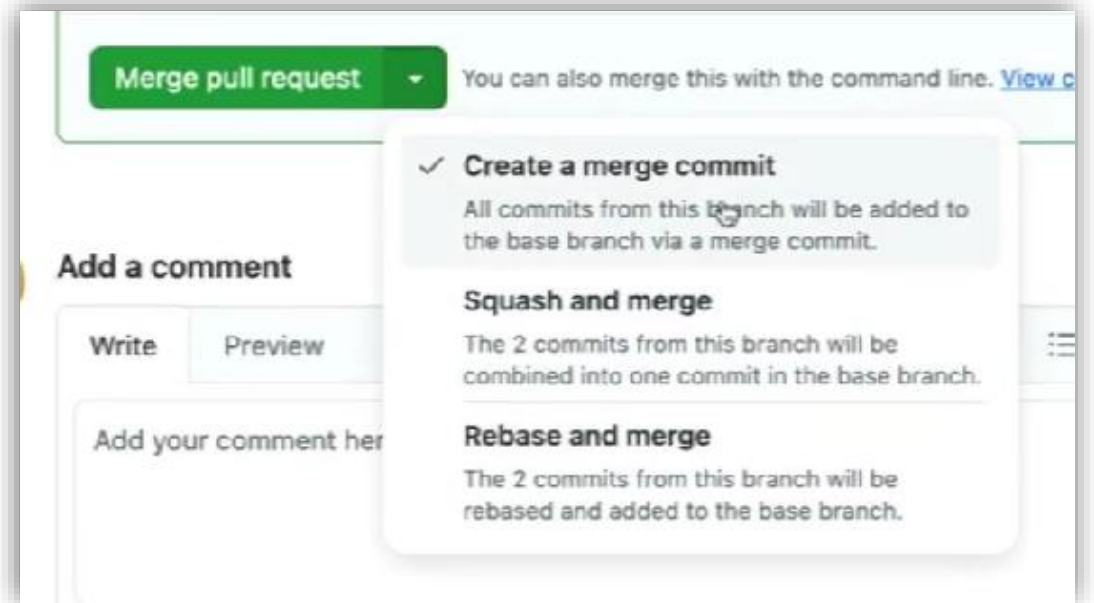
如果同事留言說我做的某個部分有問題

但我看了卻沒問題

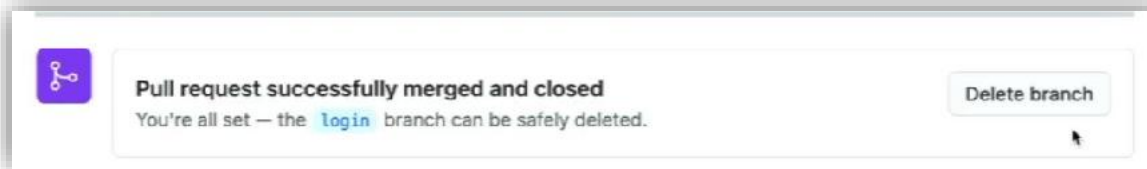
我可以按這邊回覆留言



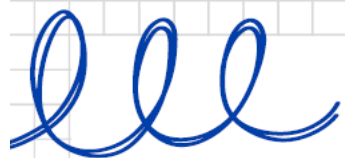
都檢查好了之後滑到最下面
找到這個綠色區域
這邊就是操作合併的地方



第一個是merge
第二個是把分支上所有異動合併成一個commit
第三個是rebase
通常會使用第一個或第三個



合併完成後就可以把剛合併完成的遠端功能分支刪掉
不會刪到本機的分

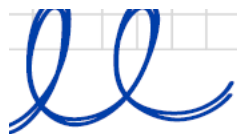


PR 注意事項



- 確認功能正確再發
- 描述清楚這個 PR 做了什麼
- 附截圖或測試結果
- 一個 PR 只做一件事
- 異動不要太大
(10 files & 400 lines 以內)
- 發 PR 前先 rebase
(確保與最新 dev 同步)



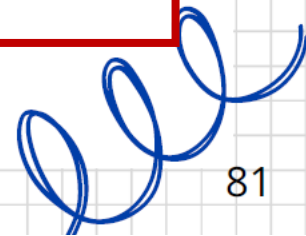


功能做到一半 突然要去其他分支怎辦？



雖然可以這樣做但太麻煩了，每一次都要想commit訊息，
而且容易忘記reset

先 add →
commit **wip** (Work In Progress) →
再reset



git stash

將當前工作區的變更暫時儲存起來，
還原成乾淨的工作區

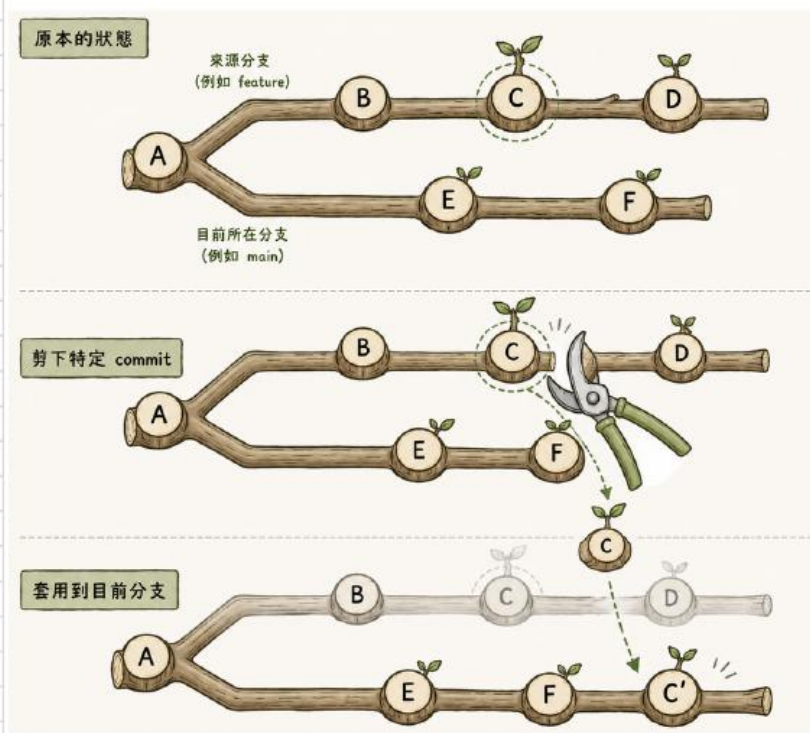
可以在不同分支解開來用
(可能會需要解衝突)
跨分支用的很方便
→ 測試dev跟優化的報告



git stash 基本指令

指令	說明
<code>git stash</code>	暫存修改，只含追蹤中的檔案
<code>git stash -u</code>	暫存修改，包含未追蹤檔案 新檔案
<code>git stash list</code>	查看所有暫存清單
<code>git stash pop</code>	套用最新 stash 並刪除該 stash
<code>git stash pop stash@{n}</code>	套用指定 stash 並刪除
<code>git stash branch <分支名> stash@{n}</code>	用指定 stash 建立新分支並套用

git cherry-pick <commit SHA>



- 從其他分支挑選指定的 commit 複製到目前分支
- SHA 值會不同，但內容一樣
- 遇到衝突時需要解衝突後
git cherry-pick --continue

Q: git rebase & git cherry-pick 差異？

lll

git cherry-pick SHA值

將feature分支上的某個 Commit 複製並合併到本地 **dev** 分支中

1. checkout到本地dev
2. git cherry-pick 2f17 (2f17是我想複製的那顆 commit 的 SHA，在feature分支上)
3. 解衝突 保留兩者/擇一/捨棄兩者
4. Ctrl+S
5. git add 檔名
6. git cherry-pick --continue
7. 終端機輸入：「:wq」

執行前的 Git Graph

假設 2f17 原本是在 feature 分支上，而你的 dev 分支原本落後或在另一個時間線上：

text

```
      C1 --- C2 (2f17) <-- feature 分支
      /
A --- B --- D          <-- dev 分支 (目前在這裡)
```

請謹慎使用程式碼。



執行後的 Git Graph

執行 git cherry-pick 並解決衝突後，歷史紀錄會變成這樣：

text

```
      C1 --- C2 (2f17)          <-- feature 分支 (完全沒變)
      /
A --- B --- D --- C2' (e4a8) <-- dev 分支 (多了一顆新 Commit!)
```

請謹慎使用程式碼。

